DESIGN AND IMPLEMENTATION OF A SMART WEB-BASED QUEUING MANAGEMENT SYSTEM WITH MOBILE TRACKING AND STATISTICAL REPORTING FOR BPL-LIEO OF LGU ROSARIO, LA UNION

Project Overview

The proposed project, entitled "Design and Implementation of a Smart Web-Based Queuing Management System with Mobile Tracking and Statistical Reporting," is intended for the Business Permits, Licensing–Local Investment and Enterprise Office (BPL-LIEO) of the Local Government Unit (LGU) of Rosario, La Union. The system aims to improve client flow, organize queue operations, reduce waiting-related problems, and support efficient service delivery.

Manual or localized queuing processes may contribute to crowded waiting areas, uncertainty regarding waiting times, skipped queue numbers, client no-shows, and uneven workloads among service counters. The proposed system addresses these concerns by centralizing queue information, enabling clients to monitor queue status, assisting staff with queue operations, and providing administrators with statistical reports.

The intended users are BPL-LIEO staff, citizens or clients visiting the office, and administrators. Information technology experts and system evaluators will also be involved in assessing the system's technical quality.

Major System Features

- ### Queue Ticket Generator

  The system will generate a unique queue number and a scannable QR code based on the selected transaction type and priority group. Tickets may be printed or accessed digitally, depending on the implemented configuration.

- ### Mobile Queue Tracking

  Clients will be able to scan the ticket's QR code using a smartphone to view their queue position and status. This function is intended to reduce the need for clients to repeatedly approach staff for queue updates.

- ### Staff Dashboard

  Authorized staff will be able to call the next client, mark transactions as completed, skip or remove tickets when necessary, and manage priority or returning clients through a centralized dashboard.

- ### Adaptive Counter Allocation and Queue Auto-Balancing

  The Smart Queue Engine will evaluate queue conditions to help distribute workload among available service counters. Factors identified in the presentation include transaction type, counter availability, priority, elapsed waiting time, and queue status. The objective is to maintain priority rules while reducing uneven workloads and p reventing regular clients from waiting indefinitely.

- ### Waiting-Time Prediction

  The system will use a Phantom Queue Simulation to estimate a client's expected waiting time. The simulation inserts an invisible or simulated ticket into the current queue matrix to determine its estimated rank. The model considers possible no-shows through an abandonment discount and calculates expected waiting time across actively staffed counters using a rolling average of service time. The prediction is an estimate rather than a guaranteed service time. Its accuracy will be assessed using Mean Absolute Error (MAE).

- ### SMS and Web Push Notifications

  The proposed system includes automated SMS and web push alerts intended to notify clients as their turn approaches. Availability of these notifications depends on the configuration and successful testing of the required services.

- ### Public Queue Monitor

  A public-facing monitor will display the current queue number and succeeding queue information so clients can follow queue progress while waiting.

- ### Statistical Reporting

  The reporting module will summarize queue transactions, office trends, service-level agreement (SLA) indicators, and employee performance measures. These reports are intended to support monitoring and operational decision-making.

Project Scope and Limitations

The proposed system covers queue-ticket generation with QR codes; transaction and priority classification; staff-controlled queue operations; adaptive queue auto-balancing and priority queuing; waiting-time prediction; public real-time queue display; mobile queue tracking; SMS and web push notifications; statistical reporting and SLA monitoring; and role-based access with cloud/local synchronization.

The target environment is BPL-LIEO of LGU Rosario, La Union. Staff will use the ticketing and queue-management dashboard, clients will obtain tickets and track queue status, and administrators will review reports and service trends. IT experts and system evaluators will assess the system's technical quality. The proposed environment is

hybrid, involving cloud-hosted and local components as determined during implementation.

The system is focused on queue management and service-flow monitoring for BPL-LIEO transactions. Supported transaction examples include new business registration, business permit renewal, and other licensing-related transactions. Priority groups identified in the presentation include persons with disabilities (PWDs), senior citizens, pregnant clients, and returning clients.

Queue status values may include Waiting, Serving, Done, Return Tomorrow, and Priority Waiting. The system is not intended to replace the entire business-permit processing or eBOSS transaction workflow. Its primary function is to support the queue and service-flow layer, including ticketing, queue tracking, staff queue control, client notifications, and reporting.

Software Development Methodology

The project will follow the Agile Software Development Life Cycle (SDLC). Agile supports iterative development, repeated testing, feedback, and review, allowing the team to refine system functions throughout the development process.

Planning

- The team will identify user needs, establish the project scope and objectives, and prioritize system features.

Design

- The team will prepare the system architecture, user interface and user experience designs, entity-relationship diagram (ERD), Data Flow Diagrams (DFD) Levels 0 and 1, use-case model, and interface wireframes.

Development

- The team will develop the frontend, backend, database and synchronization components, notification functions, and real-time communication features.

Testing

- The system will undergo unit, integration, stress, alpha, and beta testing. Identified defects will be documented, corrected, and retested iteratively.

Deployment and Review

- The system will be deployed in the planned hybrid environment and evaluated through trials and user feedback. Software quality will be assessed using the ISO/IEC 25010 characteristics identified for the project. Review findings may guide further improvements.

Software Requirements

➢ Functional Requirements

1. Generate and print queue tickets with QR codes.
2. Allow clients to select transaction types and priority groups.
3. Allow authorized staff to call, complete, skip, remove, and manage tickets.
4. Apply adaptive queue auto-balancing and priority rules.
5. Estimate waiting time through Phantom Queue Simulation.
6. Display live queue information on a public monitor.
7. Send SMS and web push notifications when configured.
8. Allow clients to track queue status using mobile devices.
9. Generate statistical reports, SLA indicators, transaction history, and employee performance metrics.

➢ Non-Functional and Quality Requirements

Functionality: The system should provide complete, correct, and appropriate functions for its intended use.

Reliability: The system should support availability, fault tolerance, and recovery.

Usability: The system should be learnable and operable, with accessible and appropriate interfaces.

Efficiency: The system should consider response time, resource utilization, and capacity.

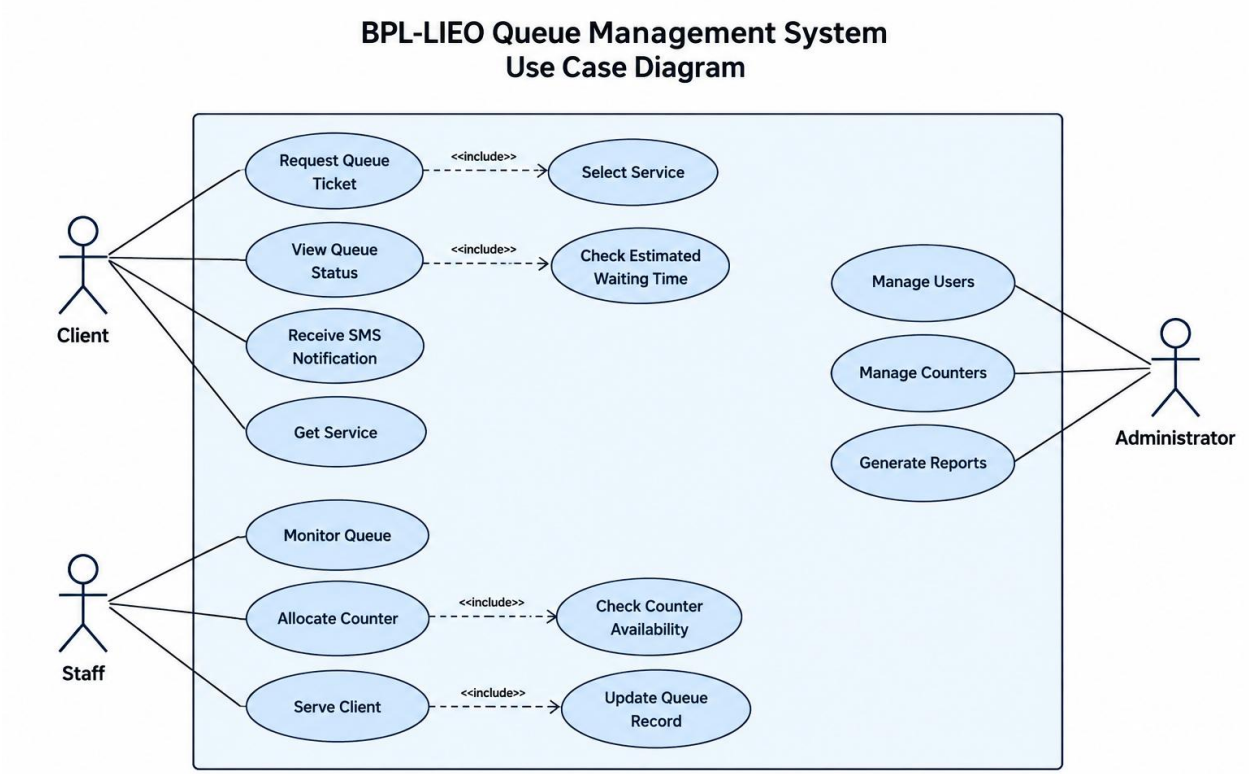
Maintainability: The system should support modularity, reusability, analyzability, modifiability, and testability.

Portability: The system should support adaptability, installability, and replaceability where applicable.

Compatibility: The system should support coexistence and interoperability with relevant components.

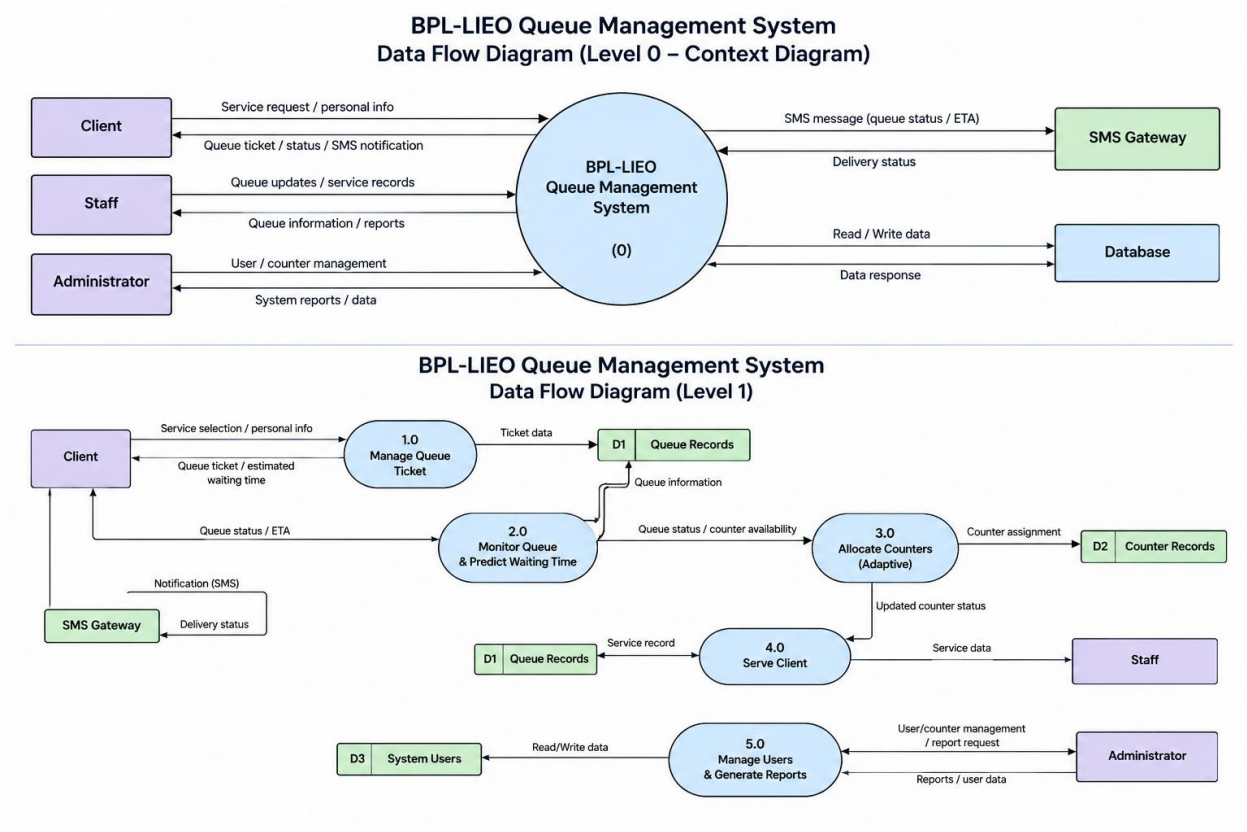
Security: The system should protect information and support confidentiality, integrity, non-repudiation, accountability, and authenticity.

System Models



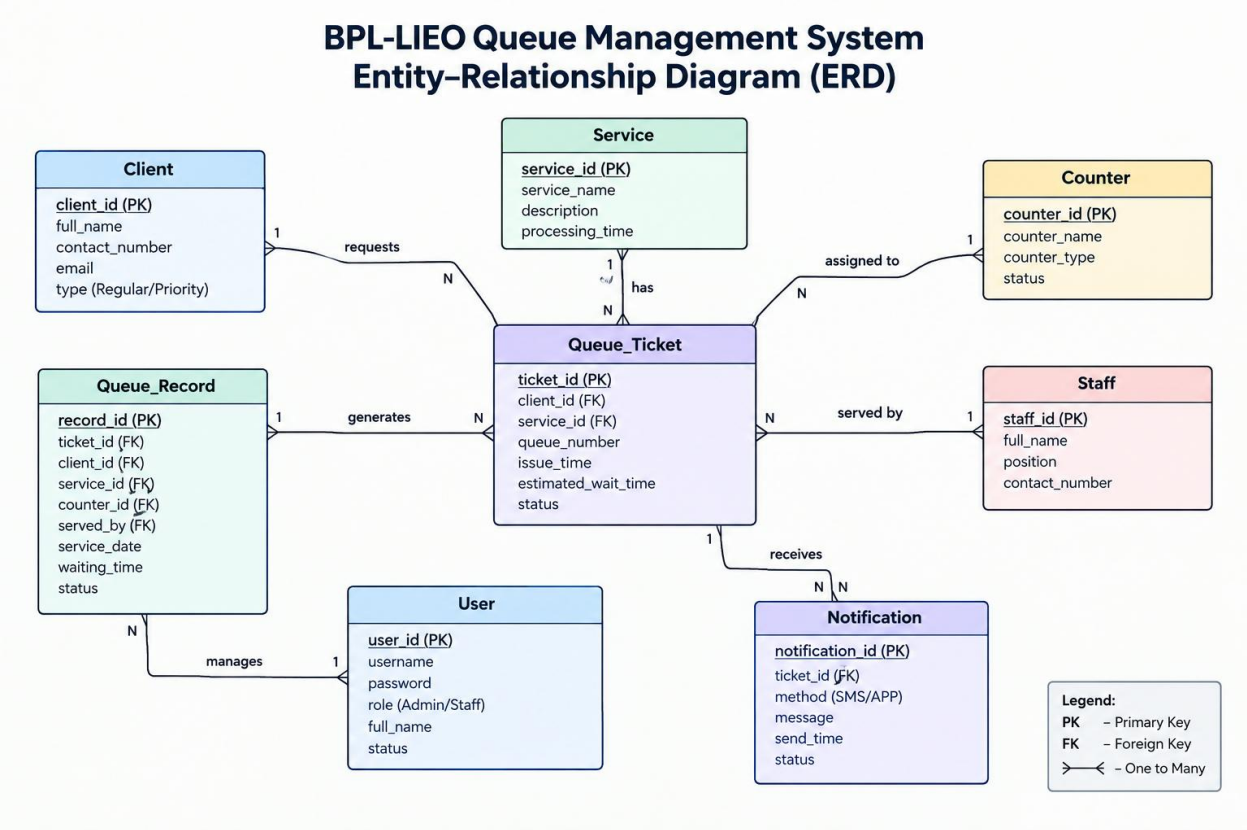
Use-Case Model

The Use-Case Model illustrates the interactions between the system and its primary users, namely the clients, BPL-LIEO staff, and administrators. Clients can obtain queue tickets, select transaction and priority types, and monitor their queue status through mobile devices. BPL-LIEO staff can manage queue operations by calling the next client, updating ticket statuses, and handling priority or returning clients. Administrators can access statistical reports, transaction histories, and service performance indicators. This model provides an overview of the system's main functions and the responsibilities of each user.



Data Flow Diagrams

The Data Flow Diagrams illustrate how information moves between external entities, system processes, and data stores within the proposed Smart Web-Based Queuing Management System. The DFD Level 0 presents a high-level overview of the system and its interactions with clients, BPL-LIEO staff, and administrators. The DFD Level 1 provides a more detailed representation of the system's internal processes, including ticket generation, queue management, waiting-time prediction, adaptive counter allocation, mobile queue tracking, notifications, and statistical reporting. These diagrams help explain how transaction details, queue updates, and report data are processed and exchanged throughout the system.



Entity-Relationship Diagram and Data Persistence

The Entity-Relationship Diagram presents the planned structure of the system's database and the relationships among its main entities. These include user accounts, queue tickets, transaction types, priority information, service counters, queue history, notifications, and reports or analytics. The ERD serves as a guide for organizing and maintaining data consistently, supporting queue operations, tracking transaction progress, recording service history, and generating statistical reports. Prisma ORM is identified as the planned database-access tool for managing interactions between the application and the database.

8. Technical Architecture and Development Tools

The proposed frontend technologies are React.js and Vite, with Tailwind CSS for styling, Framer Motion for interface animations, Recharts for data visualization, React Query for data-fetching workflows, and QR Code React for QR-code generation.

The proposed backend technologies are Node.js and Express.js. REST APIs will support communication between system components. JWT is identified for token-based authentication, bcrypt for password hashing, Zod for data validation, and Swagger UI for API documentation.

The planned data and cloud technologies include Prisma ORM, SQLite or PostgreSQL, and Firebase or Firestore. Socket.io is identified for real-time communication, while the Vonage Server SDK and Web Push are listed for notification functionality. Visual Studio Code is the development environment, Git is used for version control, and Chrome, Edge, or Safari may be

used for browser testing. Final configuration and successful integration must be confirmed during implementation and testing.

Smart Queue Logic

➢ Adaptive Auto-Balancing

The Smart Queue Engine is designed to evaluate several factors when determining queue order and counter allocation. The factors identified in the presentation are base priority weight, elapsed waiting time, configurable aging rate, SLA Panic Bonus, Stoplight Skip Limit Bonus, and Zipper Force Bonus. These factors are intended to distribute work among available service counters while respecting priority groups and reducing the likelihood that regular clients are left waiting indefinitely.

➢ Predictive Waiting Time

The Phantom Queue Simulation estimates a client's position by inserting a simulated ticket into the current queue matrix. It accounts for possible no-shows through an abandonment discount and estimates waiting time across actively staffed counters using a rolling average of service time. Prediction performance will be evaluated using Mean Absolute Error (MAE), which measures the average absolute difference between predicted and observed waiting times. A lower MAE indicates predictions closer to the observed results.

System Demonstration Workflow

The intended demonstration begins with the client selecting a transaction type and priority group, after which the system generates a queue ticket and QR code. The client then scans the QR code to view queue position, status, and predicted waiting time. On the staff dashboard, authorized personnel call the next client, skip or complete a ticket, and update queue status as needed.

The demonstration will then show queue information updating on the public monitor and mobile tracker. If SMS and web push have been configured, the notification process may also be demonstrated. The adaptive queue function will be explained by showing how priority, waiting time, SLA thresholds, counter availability, and queue status influence queue selection and allocation. When service is completed, staff mark the ticket as Done and the system records the history. The demonstration concludes with the administrator reviewing statistical reports, service trends, SLA indicators, transaction history, and performance metrics.

Testing and Evaluation

The testing plan includes unit testing, integration testing, stress testing, alpha testing, and beta testing. Unit testing will check individual functions; integration testing will examine communication among connected modules; and stress testing will assess system behavior under increased activity or load. Alpha and beta testing will support initial internal evaluation and

feedback from intended users or evaluators. Defects identified during testing will be recorded, corrected, and retested.

Technical measures identified in the presentation include a pass/fail test matrix, WebSocket latency, local/cloud data consistency, and waiting-time prediction accuracy measured using MAE. These measures will provide evidence of system behavior and performance once testing has been conducted.

The project will evaluate software quality using the ISO/IEC 25010 characteristics listed in the presentation: functionality, reliability, usability, efficiency, maintainability, portability, compatibility, and security. The planned respondents are 10 BPL-LIEO staff members, 50 citizens or clients, and 2 IT experts or system evaluators. These figures represent planned respondents; the actual number of participants and evaluation results should be reported after data collection.

12. Conclusion

The proposed Smart Web-Based Queuing Management System with Mobile Tracking and Statistical Reporting aims to improve queue organization and service-flow monitoring at BPL-LIEO of LGU Rosario, La Union. The system integrates ticket generation, QR-based mobile tracking, staff queue management, public queue display, notifications, adaptive counter allocation, waiting-time prediction, and statistical reporting within a planned web-based environment.

The Agile Software Development Life Cycle will guide iterative planning, design, development, testing, deployment, and review. Functional and non-functional requirements, together with the use-case model, DFDs, and ERD, will guide the system's design and implementation. Testing and evaluation will be used to assess whether the proposed system performs its intended functions and meets the identified software-quality criteria.